Custom UDP Server vs. Godot ENet:
A Cloud Benchmark of Multiplayer Server Capacity

Carlos L. Nunez
Department of Computer Science and Engineering
Shippensburg University, Shippensburg, Pennsylvania, USA

***Abstract*— Developers who build multiplayer games in Godot must choose between the engine's built-in ENet dedicated server and a custom server written outside the engine. We compare a custom C++ UDP server with Godot's ENet headless server under the same workload, NodeStrike, a 20×20 grid game in which every client is a bot that moves one tile per 20 Hz server tick. Both servers run on Google Cloud with one server VM and three client VMs. Two tests are used: a progressive ladder that adds clients from 1 to 300, run eight times per backend, and a 20-minute hold at fixed loads of 180 to 300 clients. We measure latency, packet handling (snapshots received per client per second), client limit, and server tick rate. TODO: write the results sentences from the ladder and hold runs.**

***Index Terms*—**game networking, UDP, ENet, Godot, tick rate, server capacity, benchmarking

I. Introduction

TODO: write the introduction (problem, what this paper does, the questions).

II. Background and Related Work

TODO: write the background (servers and transport, related work).

A. Ticks and snapshots

TODO: write what a tick and a snapshot are.

III. System Design

A. Godot client and ENet server

TODO: write how the Godot client and the Godot ENet server were built (goals, design decisions, problems you hit).

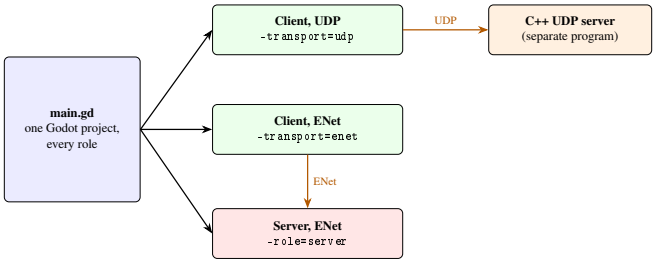


Fig. 1. The Godot project. One script is started in different modes. A UDP client talks to the separate C++ server; an ENet client talks to the same project started as an ENet server.

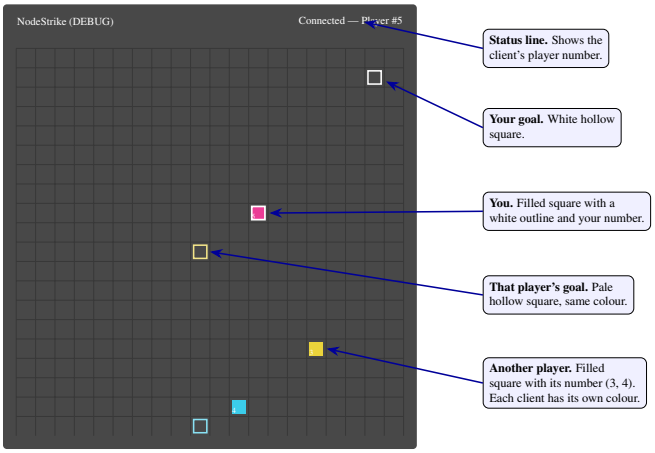


Fig. 2. The running NodeStrike client (redrawn from a screenshot of a client started in a window; headless benchmark clients draw nothing). Each tick every bot moves one tile toward its goal. The server's snapshot tells each client where every other player is; the client only draws it. Each player's pale hollow square is that player's goal.

B. C++ UDP server

TODO: write how the C++ server was built (goals, design decisions, problems you hit).

TABLE I
Messages of the C++ server protocol (version 2). Snapshots larger than 1,200 bytes are split into several datagrams.

Message	Direction	Size	Contents
JOIN	client to server	2 B	type, version
ACK	server to client	6 B	type, version, player id
STATE	client to server	18 B	position, timestamp, number, goal
TICK (snapshot)	server to client	16 + 12N B	header, then id, position and goal for each of N players

C. The two backends compared

TODO: write how the two servers work and how they differ.

IV. Methodology

All runs use one server VM and three client VMs on Google Cloud (the server on an e2-standard-2 machine, the clients on n2-standard-4 machines). ENet snapshots use the binary format from 22 September onward.

A. Main measures

Three measures are the main focus of the study. The rest of the data (CPU, memory, network use, server counters and every other

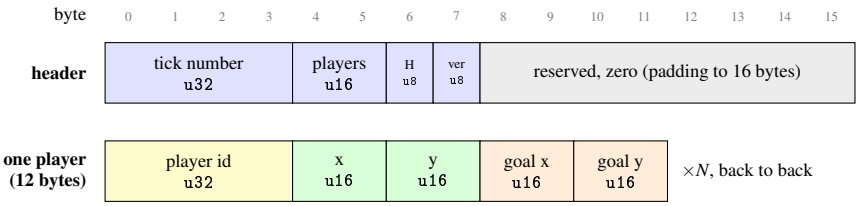


Fig. 3. Layout of one snapshot, $16 + 12N$ bytes (52 bytes for 3 players, 3 616 for 300). The ENet server writes it into one byte array and sends it as a remote procedure call; the C++ server sends the same bytes directly in UDP datagrams of at most 1 200 bytes.

TABLE II
The two backends side by side.

	C++ UDP server	Godot ENet server
Networking	raw UDP datagrams	ENet through Godot's multiplayer peer
Default port	7777	7778
Tick timing	fixed-timestep schedule: each tick is due at an exact multiple of 50 ms, so it runs at 20 ticks per second (the first version restarted its timer at each tick and drifted to about 19, see Section IV-H)	Godot timer that fires every 50 ms and keeps its timetable (20 ticks per second)
Join	JOIN message, ACK with player id	connection handled by ENet; the peer id is the player id
Position update	18-byte STATE datagram (position, timestamp, number, goal), sent every tick	`submit_state` remote procedure call (unreliable) with the same fields, sent every tick
Snapshot	16 + 12N bytes, split into datagrams of at most 1,200 B	16 + 12N bytes (binary format, header padded from 8 to 16 bytes so both sizes match), one remote procedure call
Client eviction	silent for 5 s	silent for 5 s
Latency measure	game client's ping: last send until next snapshot	same Godot client script, same measure
Telemetry	one line per second	periodic server statistics

column collected) is kept and still matters: it is used to explain why the main measures change. A tick-rate drop is the warning sign that one of the three is failing.

B. Two kinds of test

We run two kinds of test, because they answer different questions.

- **Progressive (granular) ladder.** Start with one client and add clients one step at a time, holding each step for a short time (10 s in the October runs). The state list is 1 client, then 10, 11, 12, up to 300: 292 states per backend. Question: at what client count does the server stop keeping up?
- **Hold.** Ramp up to a target client count (targets 180, 260, 270 and 300), stop adding clients, and keep them connected for 20 minutes, measured in 120 windows of 10 s. Question: does the server stay stable at a fixed load?

C. How the data is collected

Fig. 4 shows where a test runs. A Python script automates every run:

1. It creates one server VM and three client VMs on Google Cloud.
2. It starts the server (the C++ UDP server, or the Godot ENet server in headless mode).
3. It gives each client VM a plan: a list of states, each with a total client target and a duration. The client VMs start bot clients so that the total matches the plan, and each bot plays the Node-Strike game. Every bot is the same Godot client (`main.gd`): in UDP mode it talks to the C++ server and in ENet mode to the Godot server, so both servers are driven by the same client code.
4. During every state, each client counts what it sent and received, checks the snapshot sequence numbers for gaps, and measures ping about once per second.
5. The server writes one telemetry row per second (tick count, players connected, packets and bytes in and out). Each VM also records CPU, memory and network use every second.
6. When the run ends, all files are downloaded to the runner, the VMs are deleted, and analysis scripts merge the repetitions and apply pass/fail thresholds.

Gaps and incomplete snapshots are indicators of protocol-level loss, not measured network packet-loss percentages.

D. Health criteria and caveats

TODO: write the pass/fail limits and what the measurements can and cannot show.

E. Precision limits

TODO: write the precision limits.

F. Runs

Granular ramps with $N_{\max} = 300$ give 292 states per backend (1 plus 10 through 300).

G. How the runs were summed up

Both tests combine 8 complete runs per backend. For the ladder, each of the 292 client counts is paired with the same count in the other runs. For the holds, each 10 s state is paired by its position in the ramp-up or hold. The mean, lowest and highest value of each measure are reported. A ladder run passes at a client count if that

TABLE III
The three main measures, and the tick-rate warning sign. All other collected data supports them.

Measure	Plain meaning	On the charts
Latency	How long data takes to go to the server and back, in milliseconds (ping). Lower is better.	Ping in milliseconds (log scale)
Packet handling	How many packets the server receives and sends each second without losing or delaying them. A healthy client gets one snapshot per tick, about 20 per second.	Snapshots received per client per second (20 is perfect)
Client limit	The largest number of clients the server supports before performance degrades.	Share of clients still responding; where the lines break
Tick rate (warning sign)	Ticks the server completes per second. 20 Hz is healthy; below 18 Hz means the server is falling behind.	Server ticks per second, with an 18 Hz line

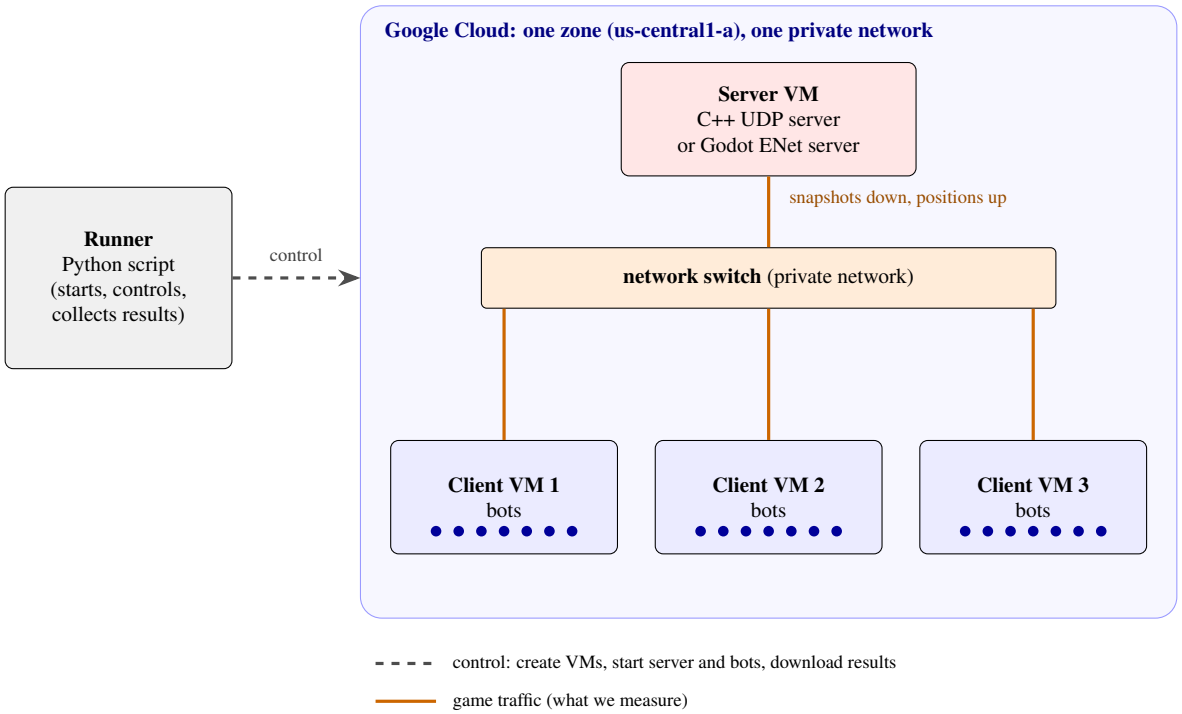


Fig. 4. Where a test runs. The runner creates one server VM and three client VMs, starts the server, tells each client VM how many bots to run, and downloads the results afterwards. The bots and the server talk to each other over the cloud's private network (a switch). Each client VM runs about a third of the bots, so the server sees all of them as one crowd.

TABLE IV
Granularity of each measured quantity. The precision limit of a conclusion is set by the coarsest quantity it relies on.

Quantity	Granularity	Where the limit comes from
Ping proxy	1 ms	The client logs whole milliseconds, but a snapshot only arrives every 50 ms (20 Hz), so differences of a few ms between backends are not reliable (limiting quantity).
Server tick rate	0.01 Hz	Computed from 9–10 samples in each 10 s window.
Snapshots per client per second	0.01	Count over a 10 s window divided by the clients.
Packet gaps	1 snapshot	Integer count of skipped snapshot numbers.
Clients responding	1 client	Whole clients.
CPU	0.1 %	Sampled once per second and averaged over the window.

count met the health limits; a hold run held if its hold averages met them. Hold run 5 needed a second attempt; its first attempt is not used.

TABLE V

Runs used in the results. The progressive ladder is eight complete runs per backend, each run a separate pass from 1 to 300 clients (one repetition each). The C++ server is driven by Godot UDP clients and the ENet server by Godot ENet clients. The hold tests are eight runs per backend and target, each a 20-minute hold (one repetition each) after a ramp-up of 10 clients per 10 s.

Date (2026)	Type	Backend	Counts	Time/state	Runs	Status
Oct 23–24	Progressive ladder (granular)	C++ server, ENet server	1, 10–300	10 s	8 runs	ok
Oct 24	Hold	C++ server, ENet server	180, 260, 270, 300	20 min hold	8 runs	ok

H. Implementation validation

TABLE VI

Problems found in the first runs.

Problem	Why it mattered	Fix	Status
C++ ticks drifted to about 19 Hz	C++ did about 5% fewer ticks per second than ENet	Fixed-timestep tick schedule	Fixed
Snapshot sizes differed (8 vs 16 header bytes)	ENet sent slightly smaller packets than C++	ENet header padded to 16 bytes	Fixed
Slow ENet states and a last-state artifact	ENet rates were measured over longer windows; the last ladder state was inflated	To be handled in the analysis	Noted

Two changes were made in `server/main.cpp`. First, the next tick is scheduled at an exact multiple of 50 ms after the previous due time, so lateness does not add up. Second, the wait for packets now ends when the next tick is due (at most 5 ms), so ticks are noticed sooner. If the server ever falls more than a whole tick behind, it resynchronizes instead of firing a burst of ticks. Fig. 5 shows the code. In a 30-second local test the original server ticked at 19.67 Hz and the fixed server at 20.000 Hz.

```
const auto tick_period =
    std::chrono::duration_cast<Clock::duration>(
        std::chrono::duration<double>(TICK_SEC));
auto next_tick = Clock::now() + tick_period;
while (true) {
    // ... wait for packets until next_tick
    //     (at most 5 ms) ...
    auto now = Clock::now();
    if (now >= next_tick) {
        next_tick += tick_period;
        if (next_tick <= now)  // a whole tick
            next_tick = now + tick_period;  // behind: resync
        // ... tick: drop silent clients,
        //     broadcast snapshots ...
    }
}
```

Fig. 5. The fixed-timestep tick schedule in the C++ server (excerpt from `server/main.cpp`). The key change is `next_tick += tick_period`: the next tick is due one period after the previous due time, not after the moment the tick was noticed.

- **Slow ENet states.** When ENet was overloaded, a "10 s" ladder state actually lasted about 27 s, because the harness waited for slow client logs. Rates are computed over the real length, so they are correct.
- **The last ladder state.** The last state (300 clients) has no later state to measure its length from, so the nominal 10 s was used while its counters covered about 27 s. This inflated its snapshot rate (up to about 30 per second, which cannot be real at 20 ticks per second).

The last ladder state is left out of the corrected ladder plots and tables (Section V-A).

V. Results

A. Test 1, progressive ladder: granular runs (both patches applied)

TODO: interpret the ladder results (latency, packet gaps, client limit, tick rate).

B. Test 2, hold at 300 clients

TODO: interpret the 300-client hold.

C. Test 2, hold at every target

TODO: interpret the holds at every target.

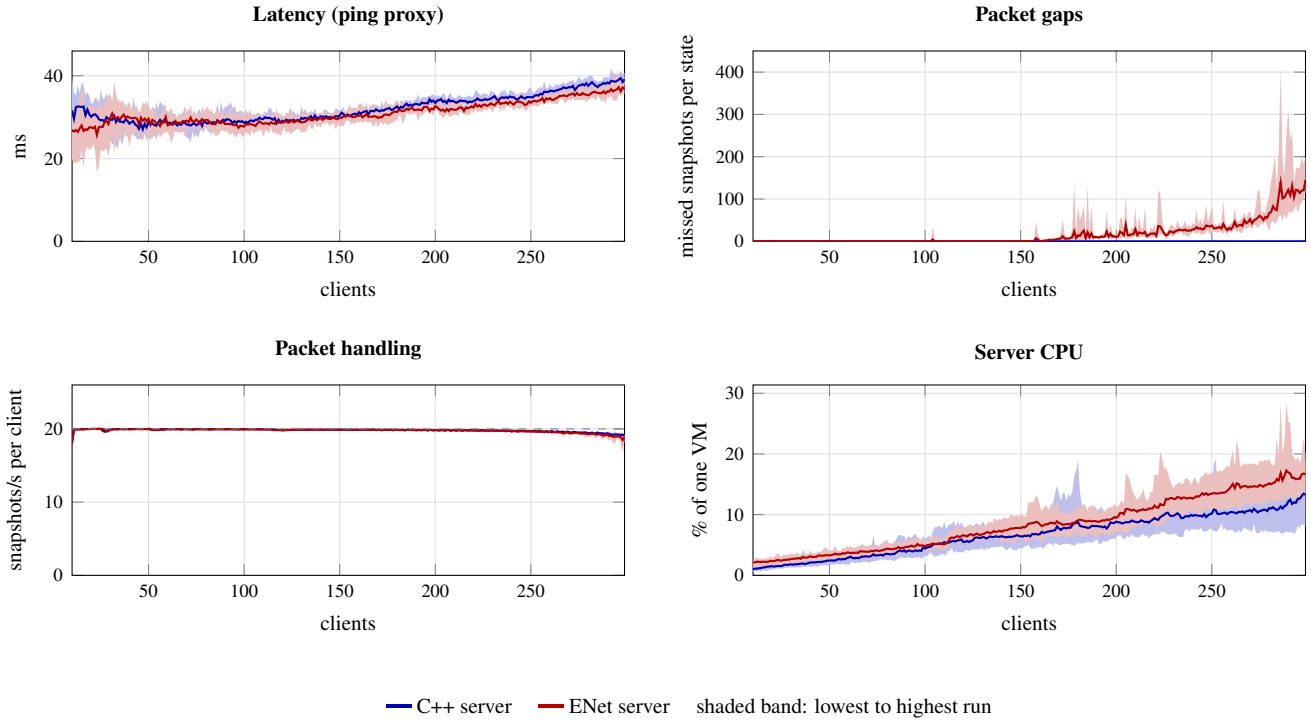


Fig. 6. Test 1, progressive ladder (granular batch, both patches applied; server on an e2-standard-2 VM, three n2-standard-4 client VMs, 10 s per client count; the 1-client warm-up state and the last state, 300 clients, are left out: the last state has no later state to time it by, so its counters cover a longer period than 10 s, see Section IV-H). Each line is the mean of 8 runs and the band spans the lowest to the highest run. The C++ server is driven by Godot UDP clients and the ENet server by Godot ENet clients. Ping is measured by the Godot client (time from its last send until a snapshot arrives), not a true round trip. Packet gaps are skipped snapshot numbers summed over all clients in a state. Dashed line: 20 snapshots per second. Clients responding was 100% in every state and the server tick rate stayed between 19.9 and 20.1 Hz for both servers (Table VII), so those two panels are left out.

TABLE VII

Ladder results at selected client counts, summed up over 8 runs (10 s per state). Ping, Snap/s and Gaps show the mean and, in brackets, the lowest and highest run. Resp. = clients responding (mean); Ping = the Godot client's ping proxy; Snap/s = snapshots received per client per second; Gaps = skipped snapshot numbers in that state; Tick = server ticks per second; Srv CPU = server CPU.

Backend	Clients	Resp. (%)	Ping, ms	Snap/s	Gaps	Tick (Hz)	Srv CPU (%)
C++ server	100	100.0	29 (26–31)	19.9 (19.9–20.0)	0 (0–0)	20.00	4.4
C++ server	150	100.0	30 (29–32)	19.9 (19.8–19.9)	0 (0–0)	20.00	6.4
C++ server	200	100.0	33 (32–35)	19.8 (19.7–19.9)	0 (0–0)	20.00	8.9
C++ server	250	100.0	35 (33–37)	19.6 (19.5–19.9)	0 (0–0)	20.00	10.2
C++ server	299	100.0	39 (38–41)	19.0 (18.8–19.3)	0 (0–0)	20.00	13.3
ENet server	100	100.0	28 (25–30)	19.9 (19.9–20.0)	0 (0–0)	20.01	4.7
ENet server	150	100.0	30 (28–31)	19.9 (19.9–19.9)	0 (0–1)	20.01	7.8
ENet server	200	100.0	33 (32–34)	19.8 (19.8–19.9)	10 (7–16)	20.00	9.5
ENet server	250	100.0	33 (33–34)	19.7 (19.5–19.8)	38 (20–85)	20.00	13.5
ENet server	299	100.0	37 (36–40)	18.9 (18.4–19.1)	145 (116–201)	19.99	16.6

TABLE VIII

Per run, over the ladder (291 states, 1 and 10 to 299 clients): states that met the health limits, snapshot gaps summed over all states, and the first client count (from 10 upward) at which a state failed (– = none).

	C++ server			ENet server		
Run	Passed	Gaps	First fail	Passed	Gaps	First fail
1	291	0	–	290	4,257	–
2	291	0	–	290	3,807	–
3	290	0	–	287	5,367	294
4	290	0	–	290	6,951	–
5	290	0	–	291	4,722	–
6	290	0	–	290	4,267	–
7	291	0	–	290	5,847	–
8	290	5	–	291	4,710	–

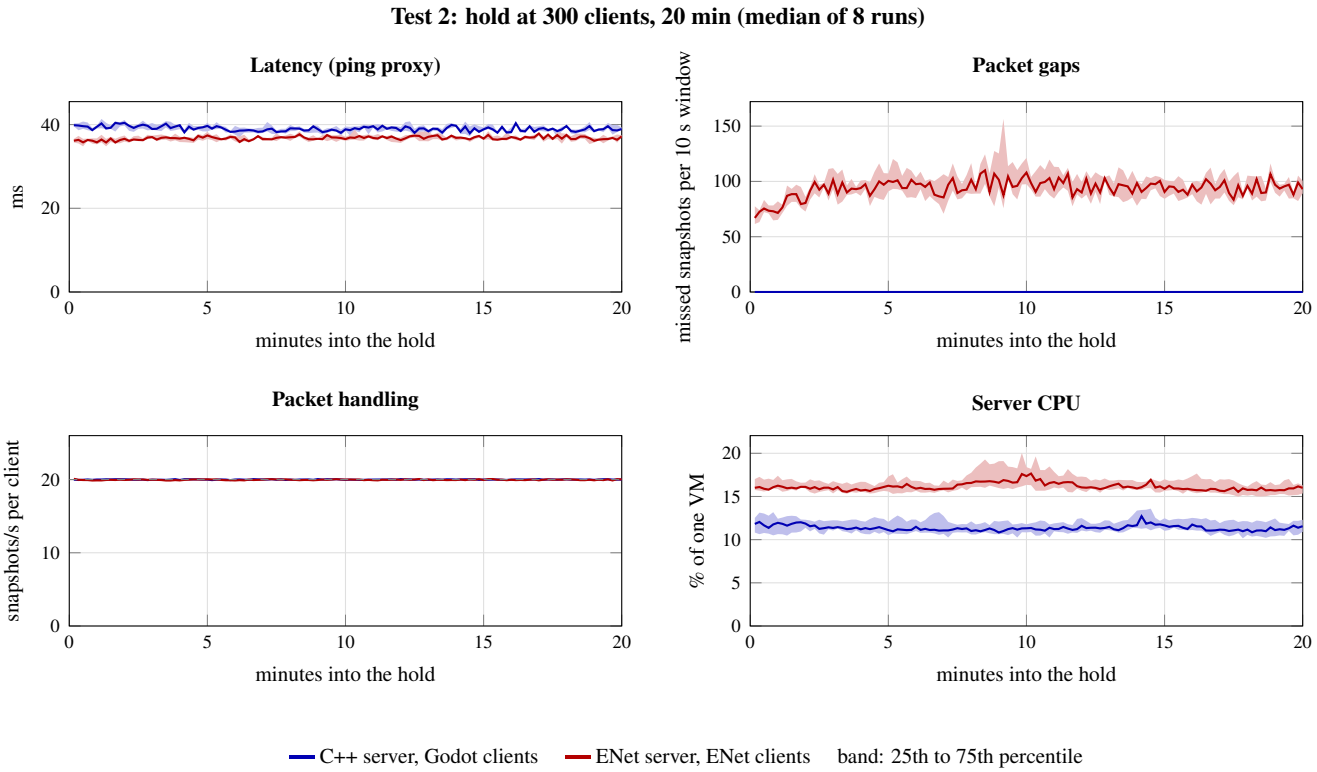


Fig. 7. Test 2, hold at 300 clients (server on an e2-standard-2 VM, three n2-standard-4 client VMs). The clients are added first (ramp-up, 10 clients every 10 s) and the plot starts when the last client has joined; it then covers the 20-minute hold with no new clients. Each line is the median of 8 runs and the band spans the 25th to the 75th percentile. Packet gaps are skipped snapshot numbers summed over all clients in a 10 s window. Ping is the Godot client's ping proxy, not a true round trip. Dashed line: 20 snapshots per second. Clients responding was 100% throughout and the median server tick rate stayed between 19.9 and 20.1 Hz for both servers (Table IX), so those two panels are left out.

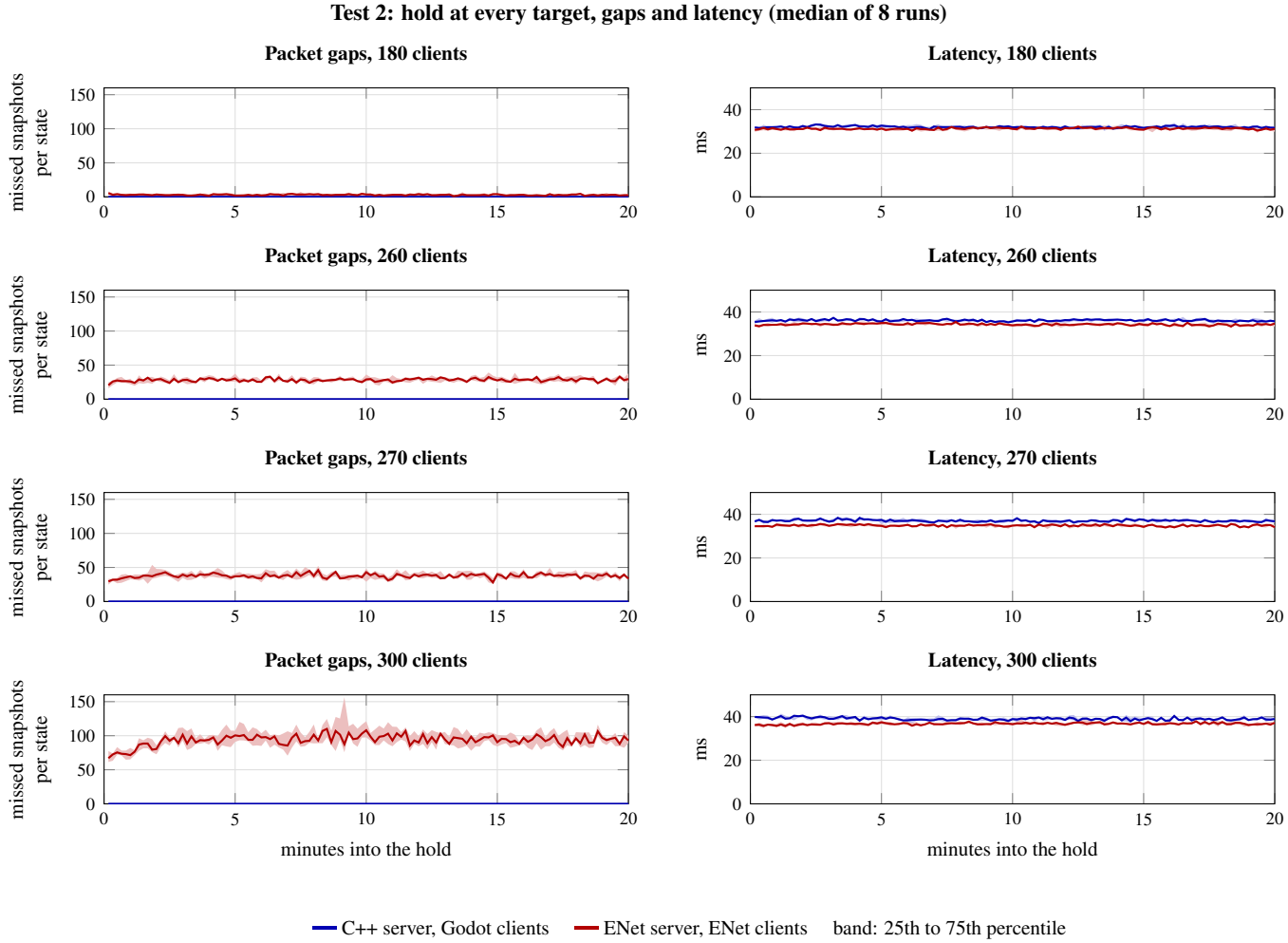


Fig. 8. Test 2, packet gaps (left) and latency (right) during the 20-minute hold at 180, 260, 270 and 300 clients. Each line is the median of 8 runs and the band spans the 25th to the 75th percentile. Gaps are skipped snapshot numbers summed over all clients in a 10 s window.

TABLE IX

Hold results at each target, from the 8 runs (C++ server with Godot clients; ENet server with ENet clients). Each run is first averaged over its 20-minute hold. Ping shows the median of the 8 run averages with the standard deviation in brackets. Gaps = skipped snapshot numbers summed over all clients and the whole hold: median with the lowest and highest run in brackets; "% of exp." is the median divided by the snapshots expected (clients × 20 per second × 1200 s). Cert. = certainty (percent) from the 8 run values: one minus the normalized fractional uncertainty (coefficient of variation divided by $\sqrt{8}$) raised to the scaling factor 0.4; n/a where the median is 0. Tick err. = absolute fractional error of the mean server tick rate against the 20 Hz baseline. Every run at every target responded at 100% and received 20 snapshots per client per second (19.96 to 20.00). p = exact two-sided Mann–Whitney U test of the 8 ENet gap totals against the 8 C++ totals ($U = 0$ at every target: every ENet run had more gaps than every C++ run; 0.00016 is the smallest p possible with 8 runs per backend). A run "held" if its hold averages met the health limits (at least 99% responding, at least 18 snapshots per second, ping at most 150 ms, tick at least 18 Hz).

Server	Clients	Ping, ms	Ping cert. (%)	Gaps (20 min)	Gaps (% of exp.)	Gaps cert. (%)	Tick err. (%)	Held	p vs C++
C++ server	180	32.1 (0.2)	91	0 (0–1)	0.0000	n/a	0.00	8 of 8	–
C++ server	260	36.2 (0.3)	90	0 (0–1175)	0.0000	n/a	0.00	8 of 8	–
C++ server	270	37.1 (0.3)	91	0 (0–0)	0.0000	n/a	0.00	8 of 8	–
C++ server	300	39.1 (0.4)	90	0 (0–8)	0.0000	n/a	0.00	8 of 8	–
ENet server	180	31.4 (0.5)	87	465 (353–830)	0.0108	59	0.01	8 of 8	0.00016
ENet server	260	34.4 (0.4)	89	3372 (3004–3771)	0.0540	76	0.00	8 of 8	0.00016
ENet server	270	34.9 (0.2)	91	4756 (3867–5031)	0.0734	76	0.00	8 of 8	0.00016
ENet server	300	36.9 (0.4)	89	11391 (9659–14963)	0.1582	70	0.00	8 of 8	0.00016

VI. Discussion

TODO.

A. Why more clients means much more work

Each snapshot lists every player, and every client gets one. So when the number of clients N doubles, each snapshot gets about twice as big and there are twice as many to send: the work grows about four times. Fig. 9 shows the amount of data this means for the C++ server (snapshot of $16 + 12N$ bytes sent to N clients, 20 times a second).

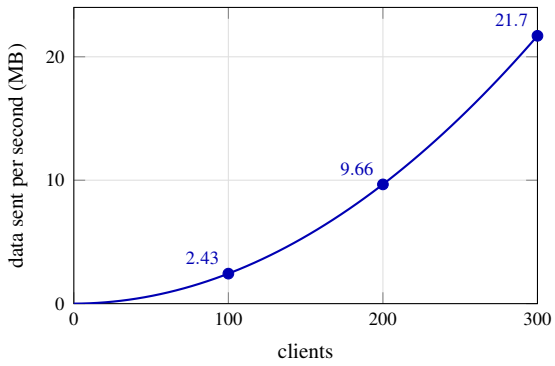


Fig. 9. Data the server must send each second as clients are added (calculated from the packet format; ENet is similar). Tripling the clients from 100 to 300 multiplies the data about nine times.

B. How clients end up responding less (and recovering)

A client counts as *responding* if it received at least one new snapshot during the measurement period. While the server keeps up, every client gets a snapshot every tick. Once the work per tick is too much, snapshots arrive late, get skipped, or pile up behind each other, and some clients stop receiving any.

C. Threats to validity

TODO: write the threats to validity.

VII. Conclusion

TODO.

References

[1] L. Aukstakalnis, "Server-client networking implementation using the Godot game engine," 2024.

[2] F. T. Al-Dhief *et al.*, "Performance comparison between TCP and UDP protocols in different simulation scenarios," *Int. J. Eng. Technol.*, vol. 7, no. 4.36, pp. 172–176, 2018.

[3] C. Jones, "Go UDP programming: A beginner-friendly guide to building fast, real-time apps," *DEV Community*, Oct. 29, 2025. [Online]. Available: `https://dev.to/jones_charles_ad50858dbc0/go-udp-programming-a-beginner-friendly-guide-to-building-fast-real-time-apps-4ik`. Accessed: Feb. 27, 2026.

[4] E. Retunsky, "Benchmarking low-level I/O: C, C++, Rust, Golang, Java, Python," *Star Gazers*, Mar. 26, 2023. [Online]. Available: `https://medium.com/star-gazers/benchmarking-low-level-i-o-c-c-rust-golang-java-python-9a0d505f85f7`

[5] Godot Foundation, "Exporting for dedicated servers," *Godot Engine Documentation*, 2026. [Online]. Available: `https://docs.godotengine.org/en/stable/tutorials/export/exporting_for_dedicated_servers.html`. Accessed: Feb. 27, 2026.

[6] Godot Foundation, "UDPServer," *Godot Engine Documentation*, 2026. [Online]. Available: `https://docs.godotengine.org/en/4.4/classes/class_udpserver.html`. Accessed: Feb. 27, 2026.

[7] J. Ahde, "Real-time Unity multiplayer server implementation," B.S. thesis, 2017.

[8] F. J. M. Arboleda, M. R. Arias, and J. A. H. Riveros, "Performance of parallelism in Python and C++," *IAENG Int. J. Comput. Sci.*, vol. 50, no. 2, pp. 1–13, 2023.

[9] A. D. Çelik and G. Şeçinti, "Network optimizing software solution for multiplayer gaming," *ITU J. Wireless Commun. Cybersecurity*, vol. 1, no. 1, pp. 47–56, 2024.

[10] GeeksforGeeks, "UDP server-client implementation in C++," Mar. 22, 2018. [Online]. Available: https://www.geeksforgeeks.org/cpp/udp-server-client-implementation-c/

[11] F. Zehra, M. Javed, D. Khan, and M. Pasha, "Comparative analysis of C++ and Python in terms of memory and time," *Preprints*, 2020, doi: 10.20944/preprints202012.0516.v1.

[12] Unity Technologies, "Ticks and update rates," *Netcode for GameObjects 2.5 Manual*. [Online]. Available: https://docs.unity3d.com/Packages/com.unity.netcode.gameobjects@2.5/manual/learn/ticks-and-update-rates.html

[13] Valve Developer Community, "Source multiplayer networking." [Online]. Available: https://developer.valvesoftware.com/wiki/Source_Multiplayer_Networking

[14] J. Gettys and K. Nichols, "Bufferbloat: Dark buffers in the Internet," *ACM Queue*, vol. 9, no. 11, 2011.

[15] M. Claypool and K. Claypool, "Latency and player actions in online games," *Commun. ACM*, vol. 49, no. 11, pp. 40–45, Nov. 2006.

[16] Y. W. Bernier, "Latency compensating methods in client/server in-game protocol design and optimization," presented at the Game Developers Conf., 2001.

[17] J. Postel, "User datagram protocol," IETF, RFC 768, Aug. 1980.

[18] L. Eggert, G. Fairhurst, and G. Shepherd, "UDP usage guidelines," IETF, RFC 8085, Mar. 2017.

[19] ENet, "ENet: Reliable UDP networking library." [Online]. Available: http://enet.bespin.org/

[20] Godot Foundation, "High-level multiplayer," *Godot Engine Documentation*, 2026. [Online]. Available: https://docs.godotengine.org/en/stable/tutorials/networking/high_level_multiplayer.html